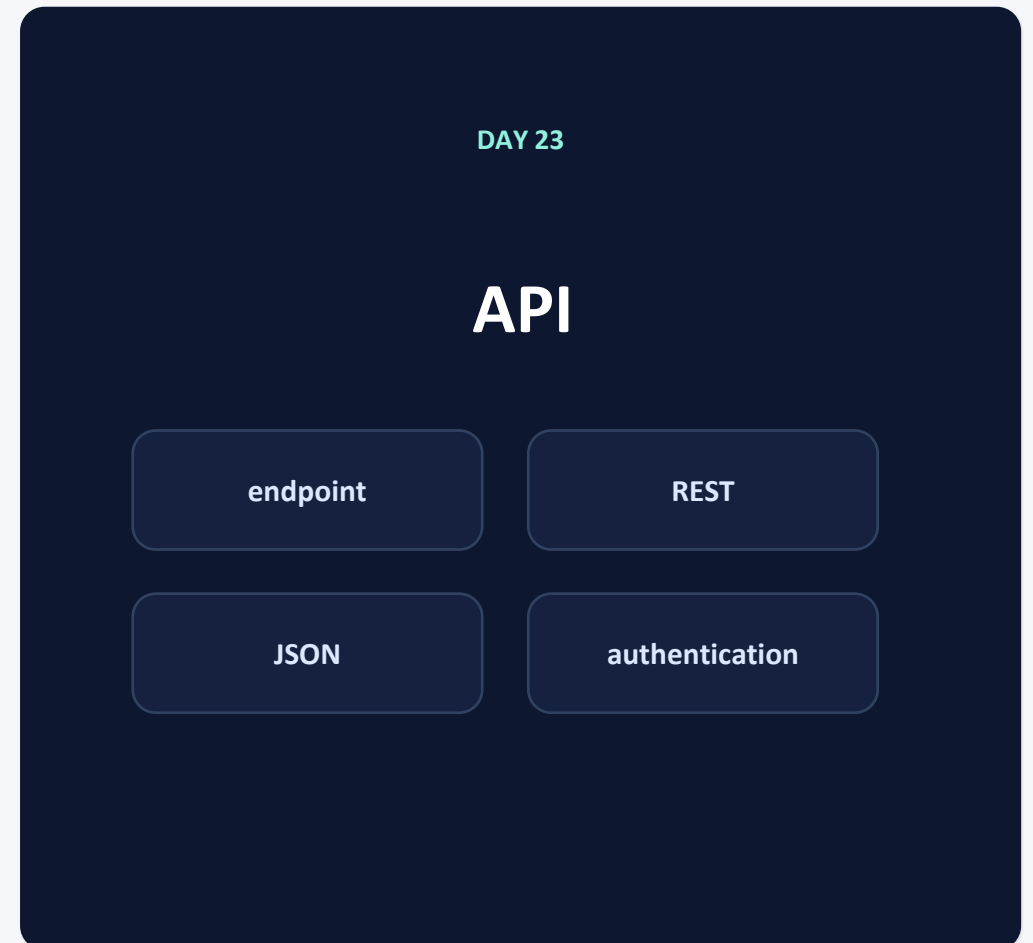


API, REST и JSON

Понимать REST API как контракт между программами: endpoint, метод, JSON, статус, аутентификация и ограничения.



ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Понимать REST API как контракт между программами: endpoint, метод, JSON, статус, аутентификация и ограничения.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример записи и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

SQL-запросы, связи и индексы

Что уже есть в нашей модели.



СЕГОДНЯ

API, REST и JSON

Вчера: SQL-запросы, связи и индексы. Сегодня мы добавляем следующий слой: api, rest и json.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Frontend, мобильные приложения и внешние сервисы часто работают с backend именно через API. Хороший контракт уменьшает зависимость между командами.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

API — официант между залом и кухней: клиент формулирует запрос по меню-контракту, API передаёт его системе и возвращает стандартизированный ответ.

ПЕРЕВОД В IT

API

согласованный интерфейс взаимодействия программ

endpoint

конкретный адрес операции веб-API

REST

стиль проектирования API вокруг ресурсов и HTTP-методов

Слой 1: API • endpoint • REST

Сначала смысл, затем профессиональный термин.

**API**

согласованный интерфейс взаимодействия программ

**endpoint**

конкретный адрес операции веб-API

**REST**

стиль проектирования API вокруг ресурсов и HTTP-методов

Слой 2: JSON • authentication • token

Сначала смысл, затем профессиональный термин.

**JSON**

текстовый формат структурированных данных

**authentication**

проверка того, кто выполняет запрос

**token**

строка, подтверждающая право клиента на запрос

Слой 3: HTTP status • payload

Сначала смысл, затем профессиональный термин.

HTTP status

числовой результат обработки API-запроса

payload

полезные данные запроса или ответа

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: rate limit • idempotency

Сначала смысл, затем профессиональный термин.

rate limit

ограничение числа запросов за период

idempotency

свойство повторного запроса не менять результат дополнительно

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day23_step1

```
{  
  "id": 1,  
  "title": "Python",  
  "days": 30  
}
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
{
```

Python выполняет эту строку сверху вниз.

```
"id": 1,
```

Python выполняет эту строку сверху вниз.

```
"title": "Python",
```

Python выполняет эту строку сверху вниз.

```
"days": 30
```

Python выполняет эту строку сверху вниз.

```
}
```

Python выполняет эту строку сверху вниз.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
{  
  "id": 1,  
  "title": "Python",  
  "days": 30  
}
```

РЕЗУЛЬТАТ

Валидный JSON-объект с тремя

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.

```
GET /api/courses/1
200 OK
{
  "id": 1,
  "title": "Python"
}
```

changed



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

Клиент запросил один курс и получил 200 + JSON

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

ОШИБКА

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.

```
{  
  'id': 1,  
  'title': 'Python',  
}
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

В JSON строки и ключи используют двойные кавычки, а после последнего поля нельзя оставлять завершающую запятую.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

ИСПРАВЛЕНИЕ

Исправленный вариант

Меняем только то, что действительно было причиной.

fixed

```
{  
  "id": 1,  
  "title": "Python"  
}
```

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Спроектировать ответ GET /api/courses/1 для существующего курса.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **GET** означает чтение.
- 2 Endpoint указывает конкретный ресурс.
- 3 Сервер находит `course id=1`.
- 4 Возвращает 200.
- 5 Тело — JSON с данными курса.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.



solution_part1

```
GET /api/courses/1
```

```
200 OK
```

```
Content-Type: application/json
```

```
{
```

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



solution_final

```
GET /api/courses/1
```

```
200 OK
```

```
Content-Type: application/json
```

```
{  
  "id": 1,  
  "title": "Python",  
  "days": 30  
}
```

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

Клиент получает однозначный статус и структурированные данные

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобрannую — но теперь первый ход за тобой.

Предложи REST-набор endpoint для создания, чтения, обновления и удаления задачи.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

POST /tasks, GET /tasks/{id}, PATCH /tasks/{id}, DELETE /tasks/{id}.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



```
POST /api/tasks  
GET /api/tasks/7  
PATCH /api/tasks/7  
DELETE /api/tasks/7
```

practice_solution

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

Четыре действия над ресурсом task

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую законченную задачу.

ЗАДАНИЕ

Спроектируй ошибку для запроса несуществующей задачи id=999.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



mini_project

```
GET /api/tasks/999
```

```
404 Not Found
```

```
{  
  "error": "task_not_found"  
}
```

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

Клиент может программно отличить отсутствие ресурса от успешного ответа

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01 Всегда возвращать 200 даже при ошибке.

02 Передавать секретный токен в URL.

03 Менять формат ответа без версии/договора.

04 Считать REST строгим набором магических URL вместо принципов работы с ресурсами.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «API» и какую проблему это понятие помогает описать?

2

Чем «API» отличается от «JSON»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 API — согласованный интерфейс взаимодействия программ.

2 API: согласованный интерфейс взаимодействия программ. JSON: текстовый формат структурированных данных.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3

Валидный JSON-объект с тремя полями

4

В JSON строки и ключи используют двойные кавычки, а после последнего поля нельзя оставлять завершающую запятую.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Понимать REST API как контракт между программами: endpoint, метод, JSON, статус, аутентификация и ограничения.
- ✓ Читать и объяснять пример темы «API, REST и JSON».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Frontend, backend и полный путь данных

Завтра: Frontend, backend и полный путь данных. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.